

无改编者 (change)

[原题链接](#)

Subtask 1

请输入文本。

Subtask 3

给一些诡异爆搜。

Subtask 6

来到正解有关环节。

首先这个转对称点的操作有个 trick 是转到差分序列上。

下面记 $d_i = a_i - a_{i-1}$ ，大概手玩一下：

将 $\{i+1, i+2, \dots, n\}$ 操作本质是将 $d_{(i,n]}$ 全部取了个相反数。

而 $\{1, 2, 3, \dots, i-1\}$ 本质是将 $d_{[1,i]}$ 全部取了个相反数，而 $d_1 \leftarrow (2 \sum_{j=1}^i d_j) - d_1$ 。

题目的限制在说，差分序列为正，所以最后 $[2, n]$ 的 d 就是其绝对值。

而答案为差分序列所有数的和，但由于保证不对 a_1 做任何修改也可以达到最优解，所以答案为差分序列上所有数的绝对值的和。

Subtask 7

不难发现每次我们都加上了 $(2 \sum_{j=2}^i d_j)$ 。无论如何修改，到最后的 d'_1 都满足条件：

令 $g = 2 \gcd(d_2, d_3, \dots, d_n)$ ， $d_1 \equiv d'_1 \pmod{g}$ 。

下面证明 d'_1 可以取到所有与初始 d_1 同余于 g 的正整数。

考虑如果操作一次 $\{1, 2, 3, \dots, i-2\}$ 和一次 $\{1, 2, 3, \dots, i-1\}$ 简单算一下，发现 d_1 加上了 $2d_{i-1}$ 。

同时用后缀操作简单调整，就可以做到让 d_1 变成所有 $d_{[2,n]}$ 的线性组合。由裴蜀定理知， d_1 可以取到所有与初始 d_1 同余于 g 的正整数。

时间复杂度 $O(T(n + \log V))$ 。

木琴修炼手册 (feedback)

性质 D 是启发性性质，故从此开始考虑。

易发现性质 D 是从小到大排序后是希望是哪种就把这道题出成哪种。

考虑观察过程，最终的方法是两种题交替出，最后希望是哪种就把大的部分给哪种，考虑往前推，发现我们是用了 $[1, n]$ ，往前推就是从后往前少了一个，也就是用的 $[1, n-1]$ ，并且情况进行了反转。

故猜测从后删会改变情况，再看从前删，发现从前删无论如何都不会改变情况，故有了从后往前推一个状态并且可以做到改变或不改变状态的做法。从后往前推就能做出来，时间复杂度 $O(n \log n)$ ，复杂度瓶颈在于排序。

暴怒豆包vs暴食蟑螂（furioso）

Subtask 1,2

有一个是人都会写的暴力 DP。

$$dp_i = \min_{j=1}^{i-1} (dp_j + w(j, i))。$$

其中 $w(l, r) = (r - l)^2 \min_{i=l}^r a_i。$

Subtask 3

性质一：每次跳跃时取得的最小值是在端点处。

若不满足上诉条件，从 l 跳到 r ，若 m 为 $[l, r]$ 中最小值的下标又满足 $l < m < r$ 。

那么从 l 跳到 m ，再从 m 跳到 r 会更优。

得证。

只保留这些合法的转移。

考虑每个 a_i 的可能转移 j ，假设 k 为最大的 $k < i, a_k = a_i。$

不难发现 $\max j \geq k$ ，所以对于相同的 a_i ，总转移量是 $O(n)$ 的。

Subtask 4

性质二：如果跳到 i ，对于上一步或者下一步总满足跳跃长度最大为 $\frac{n}{a_i}。$

假设长度为 d ，那么跳 d 次长度为一的代价最大为 dn ，而直接跳最小代价为 $a_i d^2。$

由于 $a_i d^2 < dn$ ，得证。

仅保留这个转移即可。

Subtask 6

简单计算发现对于 $a_i \leq \sqrt{n}$ 的由于性质一，复杂度为 $O(n\sqrt{n})$ ，对于 $a_i > \sqrt{n}$ 的，由于性质二，复杂度为 $O(n\sqrt{n})。$

完成。

双重承诺（promise）

原题：P16422。

记 a_i 的前缀和为 $S_i。$ 一条直线的斜率记为 k ，自变量记为 x ，截距记为 $b。$ 其余记号 n, q, A 与题目描述中相同。

Sub1

枚举最终的任务排列，按照题目公式计算答案，可以同时处理 $R = 1, 2, 3, \dots, n$ 的答案，询问时直接输出即可。时间复杂度为 $O(n \times n! + q)。$

Sub2

这一部分可以直接使用状压枚举来做，但这里主要用于说明原问题的转化过程，先看一个比较显然的结论：

- 固定检验集合 T 后，存在一种最优方案，使 T 内部按原编号递增，其余订单组成的集合 U 内部也按原编号递增。

记第 i 份订单的完成时间和完成次序相对原计划的增量分别为 D_i, E_i ，其赔偿为 $\max(D_i, 0) + a_i \max(E_i, 0)$ 。令 $H_i = D_i + a_i E_i$ ，由三角不等式可得总赔偿至少为 $\sum_i \max(H_i, 0) \geq \sum_{i \in U} H_i$ 。

对于一对原编号满足 $i < j$ ，但实际加工时 j 在 i 前面的订单， H_i 增加 $a_i + a_j$ ， H_j 减少 $a_i + a_j$ 。最优方案中策略类似：将 U 中所有订单的 H_i 相加， U 内部的贡献相互抵消，只剩下所有被迫跨越的 (i, j) ，这正好可以达到答案的下界。所以将 T, U 内部分别按编号顺序排列最优。

把未检验订单记为 0，检验订单记为 1，所以想得到某种检验集合只需不断交换相邻的 01，也就是将 1 逐步归位。若当前交换的是原订单 $i < j$ ，其中 i 未被选择， j 被选择，则这一对的贡献为 $a_i + a_j$ ；两份订单的原编号相邻时，就是 $a_i + a_{i+1}$ 。因为每一对满足 $i < j, i \notin T, j \in T$ 的订单恰好跨越一次，所以最优情况下，总赔偿就是所有这些交换代价之和。

现在已经导出了另一种做法：枚举集合 T ，按照上述方法计算相应贡献，时间复杂度可以做到 $O(n2^n + q)$ 。

Sub3

上一步的观察还可以继续优化。从最终集合的状态考虑，令 T 按编号排为 $s_1 < s_2 < \dots < s_R$ 。来自 a_{s_i} 的贡献可以写成 $(s_i - i)a_{s_i}$ ，被它跨过的未选择订单的加工时间之和为 $S_{s_i-1} - \sum_{j < i} a_{s_j}$ 。总费用可以写成：

$$\sum_{i=1}^R \left(A a_{s_i} + (s_i - i) a_{s_i} + S_{s_i-1} - \sum_{j < i} a_{s_j} \right)。$$

其中的双循环求和可以化简为： $\sum_{i=1}^R \sum_{j < i} a_{s_j} = \sum_{i=1}^R (R - i) a_{s_i}$ 。所以贡献公式直接写成： $\sum_{i=1}^R (S_{s_i-1} + (A + s_i - R) a_{s_i})$ 。

每一项都完全独立，所以对于固定的 R ，每个订单 i 被选择的独立代价为 $f_i(R) = S_{i-1} + (A + i - R) a_i$ ，只需选择其中代价最小的 R 个订单。这一步时间复杂度可以做到 $O(n^2 \log n + q)$ 。

Sub4

将 $f_i(R)$ 看成一条斜率 $k = -a_i$ 、自变量 $x = R$ 、截距 $b = S_{i-1} + (A + i) a_i$ 的直线。由于所有 a_i 相同，直线互相平行；设其公共值为 k_0 ，则截距随 i 递增，最优方案一定选择前 R 份订单，时间复杂度为 $O(n + q)$ 。

Sub5

这一档是启发性子任务，可以导出正解的所有重要思路。

维护 $sumk$ 表示所选直线的斜率之和，同时用 $sumb$ 维护所选直线的截距之和。

设 a 有两种取值，分别为 b_1, b_2 ，且 $b_1 > b_2$ 。斜率取负号，所以有 $k_1 = -b_1 < k_2 = -b_2$ 。同一组内斜率相同，所以直线之间的相对顺序永远不变。将两组截距分别排序，组内取值按顺序单调。

设两组的大小分别为 n_1, n_2 。每次询问只需在 $\max(0, R - n_2) \leq mid \leq \min(R, n_1)$ 内二分从第一组选择的直线数，并检查是否满足：

- 第一组第 mid 个已选值不大于第二组第 $R - mid + 1$ 个未选值。
- 第二组第 $R - mid$ 个已选值不大于第一组第 $mid + 1$ 个未选值。

给不存在的位置设定正负无穷极值，直接二分这个 mid 就可以得到最优解，时间复杂度为 $O(n \log n + q)$ 。

Sub6

这是给常数稍劣的正解，或基于李超树的 $O(n\sqrt{n}\log^2 n)$ 做法预留的分数。

正解

Sub5 启发我们将直线分组，并通过修正多组之间的选择得到最优解。但由于组数太多，所以一个自然的方向是**按照斜率分块**。

我们依次求出 $R = 1, 2, 3, \dots, n$ 的答案。设块长为 B ，块数为 $m = \lceil n/B \rceil$ ，取 $B \approx \sqrt{n}$ 。

将所有直线按斜率 $k_i = -a_i$ 从小到大排序，这样对于块编号 $x < y$ ，块 x 中任意直线的斜率都不大于块 y 中任意直线的斜率。这点是后续复杂度证明的核心前提。

每个块维护一个数组，始终按照当前横坐标 x 处的直线值从小到大排列，再维护当前答案集合在第 i 块中选择的直线数 cnt_i ，其中被选择的必然是一段前缀。

每次处理新的横坐标 x 时，先调整块内顺序。只需维护相邻直线的交换。若块内当前相邻直线为：

$$u(x) = k_u x + b_u, \quad v(x) = k_v x + b_v,$$

并且当前顺序为 u 在 v 前面，那么只有 $k_u > k_v$ 时， v 才可能在未来移动到 u 前面。若允许函数值相等时交换，则第一次可以交换的整数时刻为 $t = \left\lceil \frac{b_v - b_u}{k_u - k_v} \right\rceil$ ；若算出的时刻早于当前时刻，则取当前时刻。用一个可修改最小堆维护所有块内相邻位置的事件时间。处理时刻 x 前，不断取出事件时间不超过 x 的位置并交换相邻直线，同时更新相邻位置的事件时间。**注意**，若交换位置恰好位于已选前缀的边界，还需要相应修改 $sumk, sumb$ 。

处理完时刻 x 的全部块内交换后，再调整各个块之间的最优选集。使用堆 mn 维护全局最小未选直线，用堆 mx 维护全局最大已选直线。如果当前 mn 堆顶的值小于 mx 堆顶的值，就不断调整。具体而言，设二者来自块 u, v ，则：

1. 从块 v 删除最大已选直线，令 $cnt_v \leftarrow cnt_v - 1$ ；
2. 向块 u 加入最小未选直线，令 $cnt_u \leftarrow cnt_u + 1$ ；
3. 修改维护直线信息，并更新两个堆中块 u, v 的候选位置。

修正结束后，原有的 $x - 1$ 条已选直线已经是当前最小的 $x - 1$ 条。最后把全局最小未选直线加入，得到最小的 x 条直线，并更新答案。由于横坐标改变后各块候选直线的值都会改变，所以每轮需要用 $O(m)$ 的时间重建两个块级堆。

复杂度证明

对于单个块，由于两条直线最多相交一次，所以交换次数上限为 $O(B^2)$ 。共有 $m = O(n/B)$ 个块，因此块内交换总数为 $O(nB)$ ，这一部分的总时间复杂度为 $O(nB \log n)$ 。另外，每轮用 $O(m)$ 的时间重建块级堆，总复杂度为 $O(nm) = O(n^2/B)$ 。

最重要的还是**跨块修正次数**的证明，定义势能 $\Phi = \sum_{g=1}^m g \cdot cnt_g$ 。

由于块按斜率递增排列，当横坐标增加后，一条未选直线若越过一条已选直线变得更优，它所在的块编号一定更小。因此每次跨块修正都会把一个选择名额从较后的块移到较前的块，使 Φ 至少减少 1。

每轮最后新增一条直线，最多使 Φ 增加 m 。总共只有 n 轮，且势能始终非负，所以跨块修正总次数为 $O(nm) = O(n^2/B)$ 。每次修正只修改常数个块级堆节点，复杂度为 $O(\log m)$ 。

综上，总时间复杂度为 $O(nB \log n + \frac{n^2}{B} \log n)$ 。取 $B = \Theta(\sqrt{n})$ ，即可在 $O(n\sqrt{n} \log n)$ 的时间内解决本题。建议采用手写二叉堆；维护块内交换时间部分使用线段树也可能更加自然，总之实现常数还是不太小的。